SC20

Everywhere we are | more than hpc.

CAB-MPI: Exploring Interprocess Work Stealing towards Balanced MPI Communication

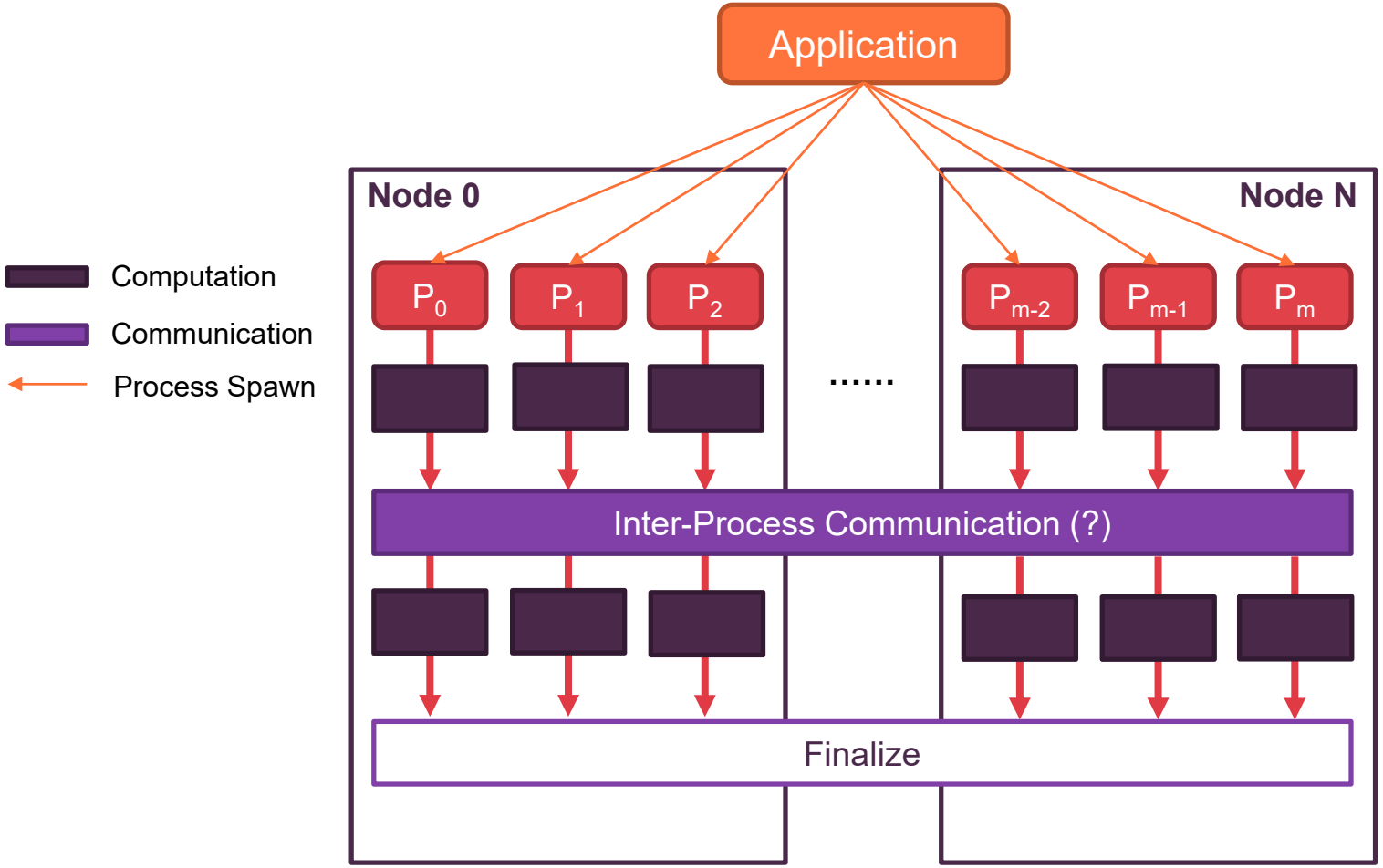
Kaiming Ouyang*^, Min Si^, Atsushi Hori†, Zizhong Chen* and Pavan Balaji^

University of California, Riverside*,
Argonne National Laboratory^,
RIKEN Center for Computational Science†

Argonne NATIONAL LABORATORY

UC RIVERSIDE UNIVERSITY OF CALIFORNIA

Is Application Really Balanced?

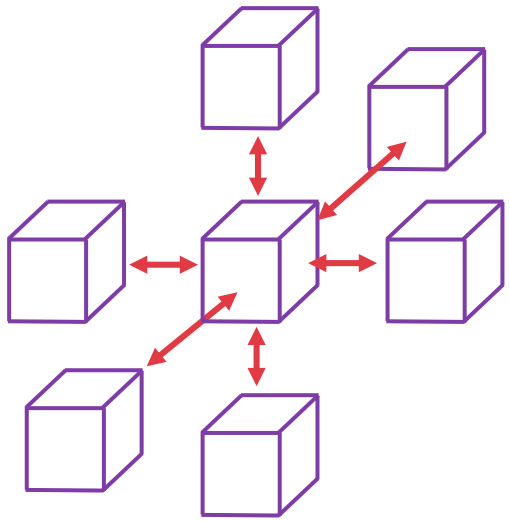


- **Computation Load Balance**
  - widely studied
  - well optimized on application level
- **Communication Load Balance**
  - not fully controlled by developers
  - can be well balanced like computation?
  - not well investigated; impact is unknown

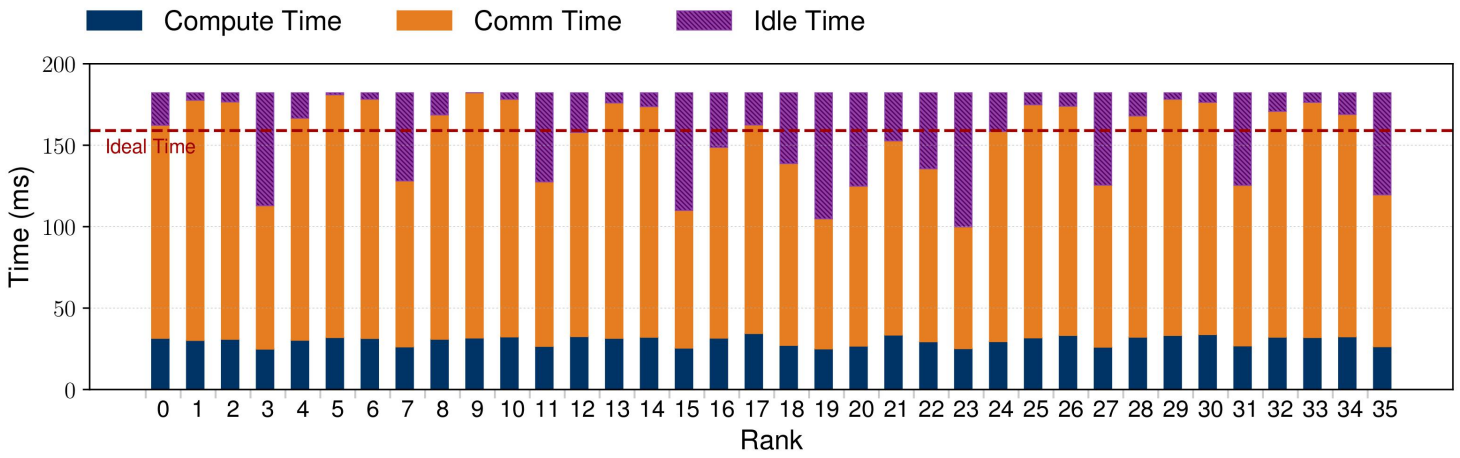
Is Communication Really Balanced?

Where is imbalance from?

- **MPI runtime**
  - communication can happen among inter-node and intra-node (intra- and inter-NUMA)
- **Application**
  - processes on the grid surface have less neighbors
  - datatype of each face is different



3D 7-point stencil process communication pattern



process grid (4x6x6) on 4 broadwell nodes

- Communication time varies a lot (communication imbalance)
- Actual runtime is around **20%** larger than ideal time

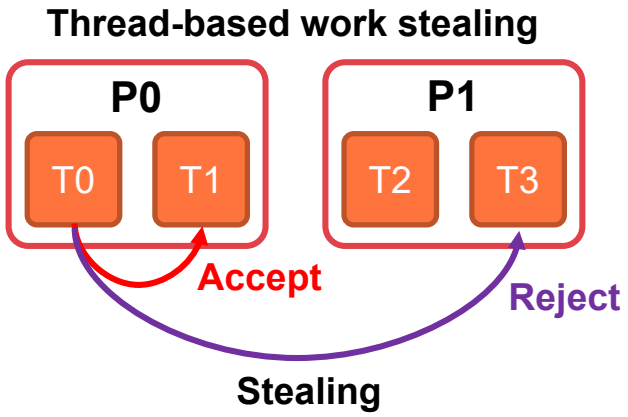
Communication Rebalance Solutions

Application-level solution

- **MPI runtime imbalance mitigation**
  - collect runtime information (communication time, workload, type and so on) and dynamically rebalance workload **(complex & not efficient)**
- **Application workload imbalance mitigation**
  - cannot solve inherent communication workload imbalance

System-level solution

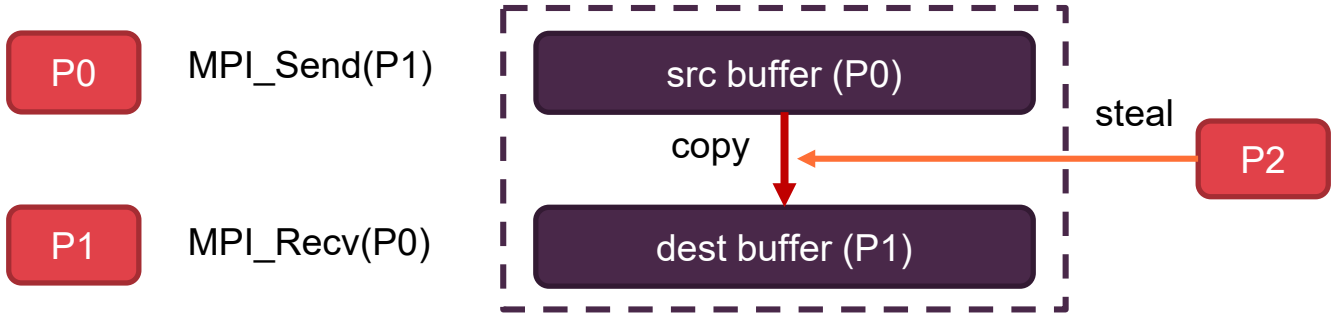
- **Thread-based work stealing**
  - widely used for computation workload imbalance
  - incapable of optimizing interprocess communication



Communication Rebalance Solutions

Interprocess work stealing

- target interprocess communication
- mitigate communication imbalance among processes



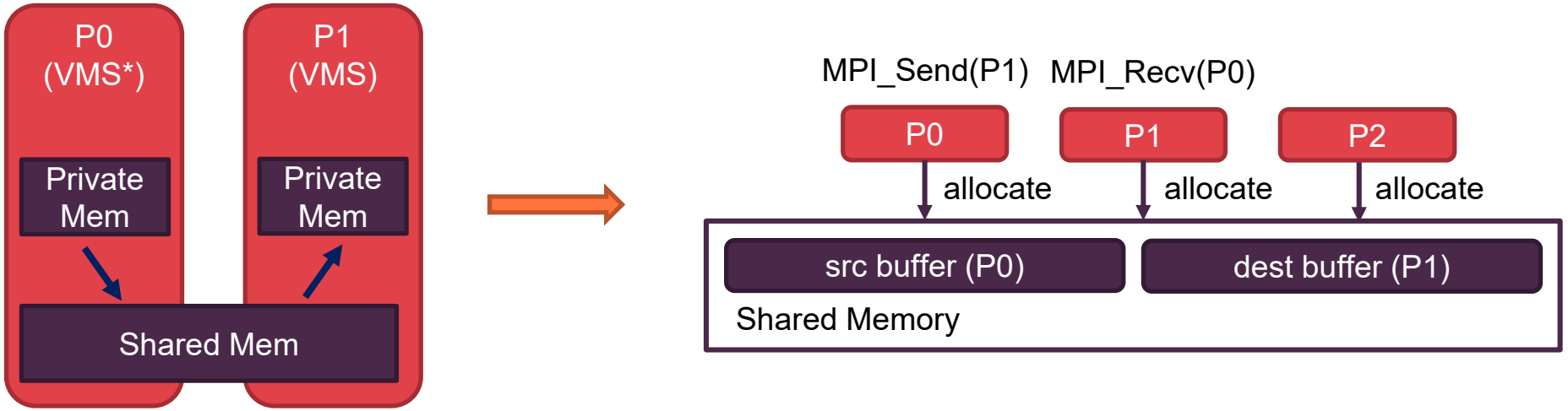
- NUMA-aware stealing to maximize performance

Interprocess Data Access Challenge

- Forbidden data access between processes by Operating System
- Need shared memory techniques
- Many shared memory techniques are applicable
  - **key factor:** low data access overhead

Potential Shared Memory Technique

POSIX-shm

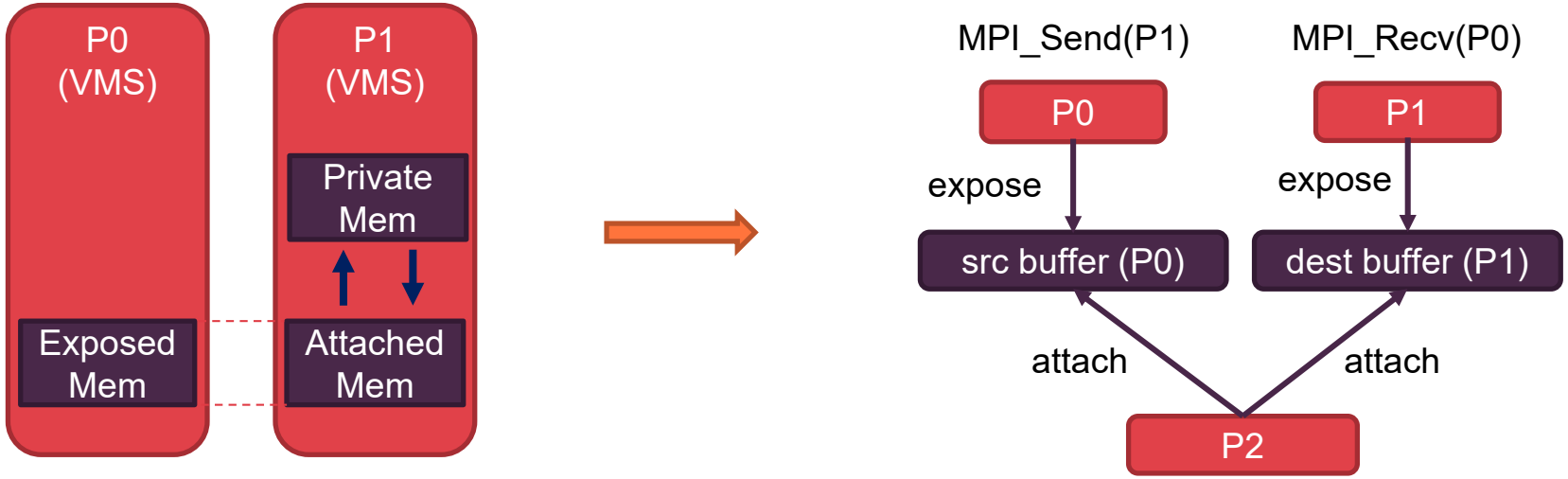


- require processes to allocate shared memory collectively **(high overhead)**

* VMS stands for virtual memory space

Potential Shared Memory Technique

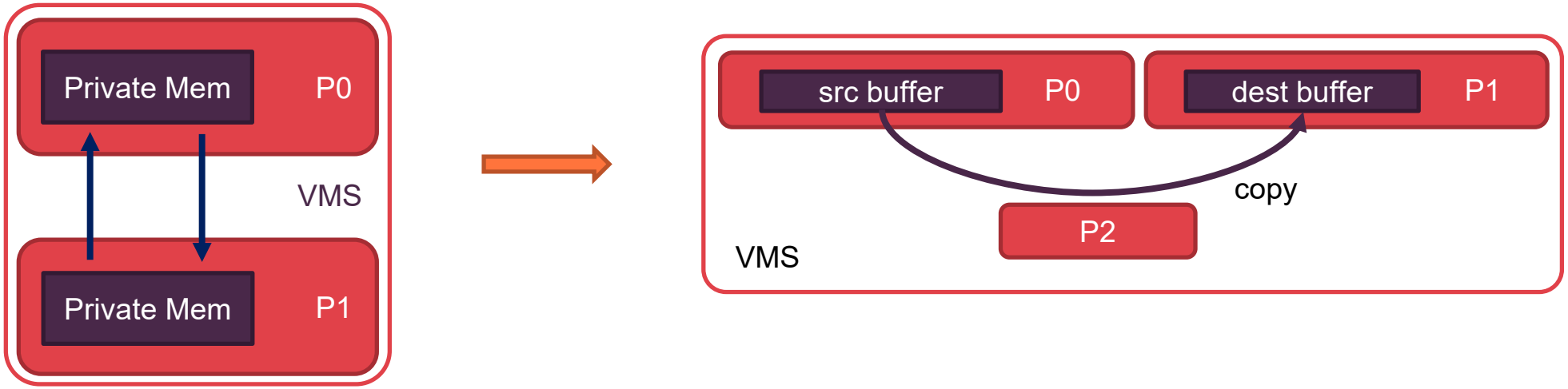
XPMEM



- system call (expose and attach) and page fault overhead can offset stealing benefits **(expensive)**

Potential Shared Memory Technique

PIP[1] (Process-In-Process)



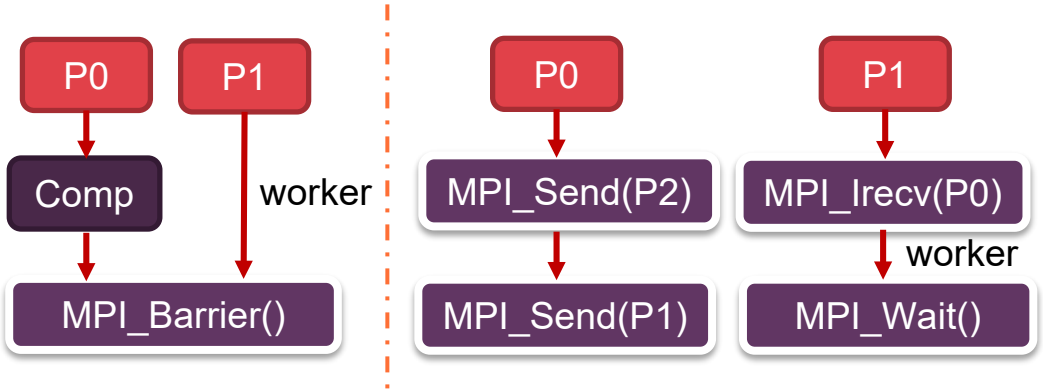
- flexible data access
- no system call and page fault overhead ☺

[1] Hori, Atsushi, et al. "Process-in-process: techniques for practical address-space sharing." Proceedings of the 27th International Symposium on High-Performance Parallel and Distributed Computing. 2018.

Definition

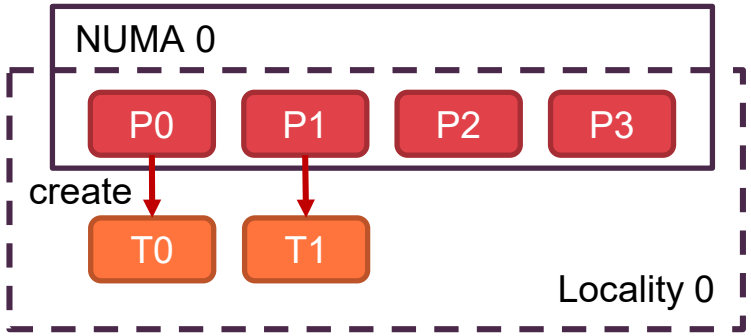
Worker

- an idle process inside MPI



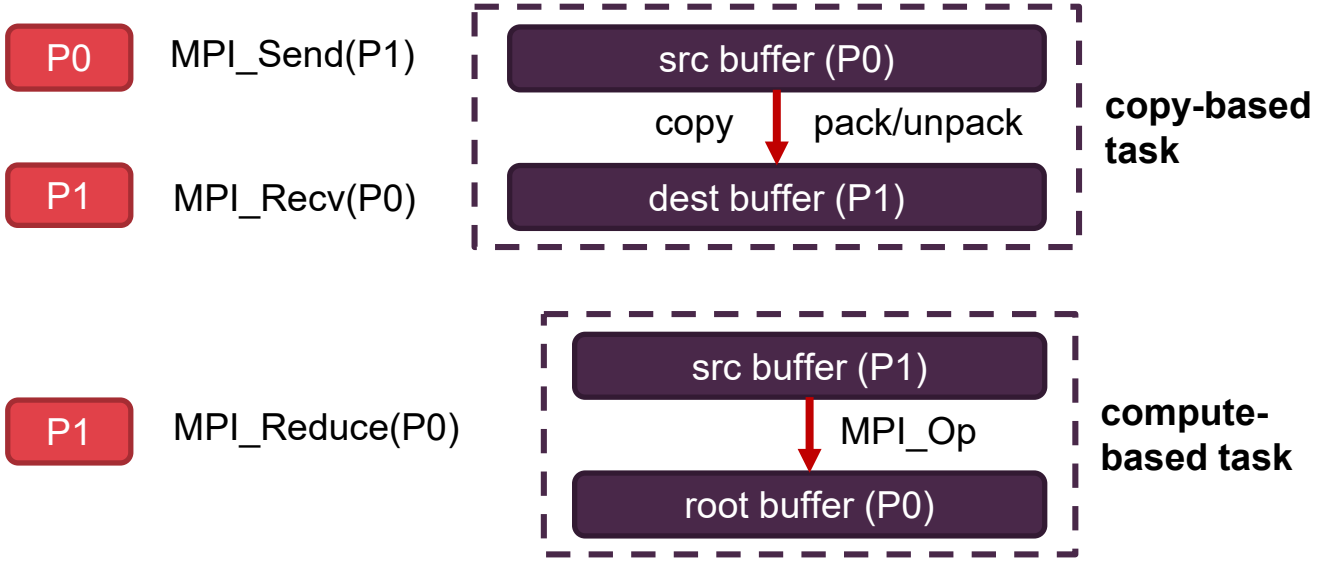
Locality

- Local Stealing: task locality = worker locality

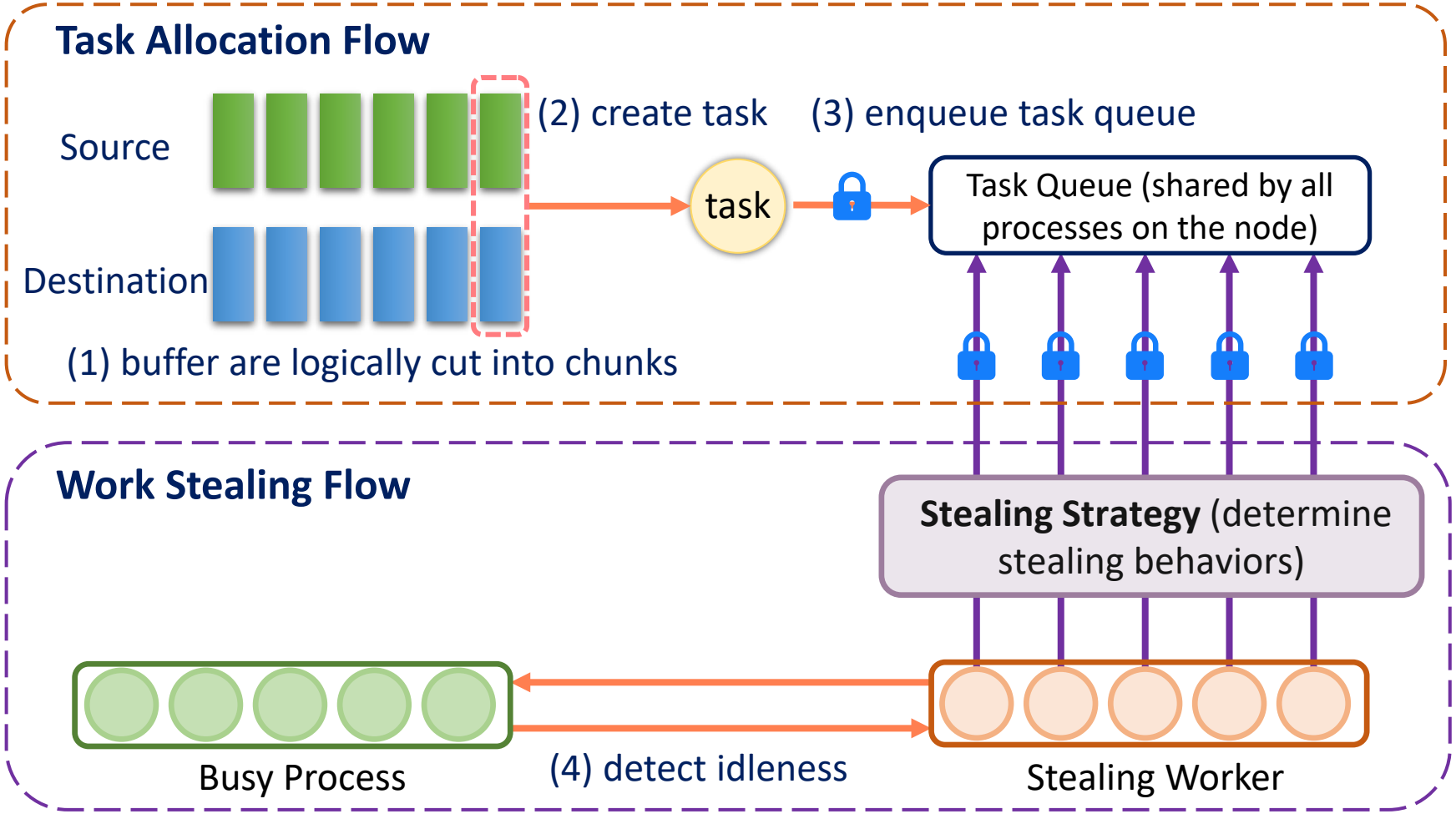


Task

- copy/compute a certain amount of data from src to dest buffer



Work-Stealing Framework



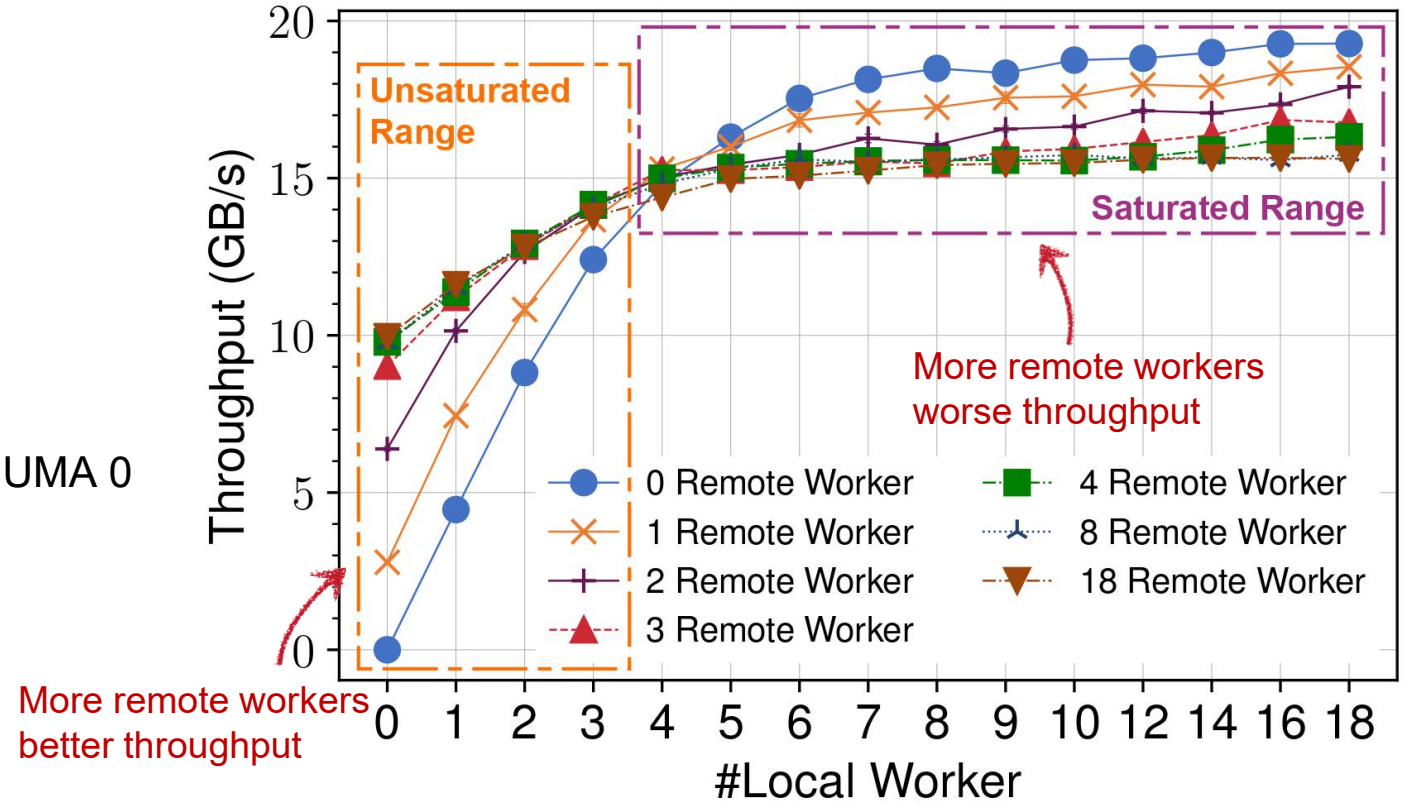
Key Factor for Stealing Performance

Task in MPI

- memory-bound
- memory bandwidth utilization is the key

Memcpy throughput Profiling

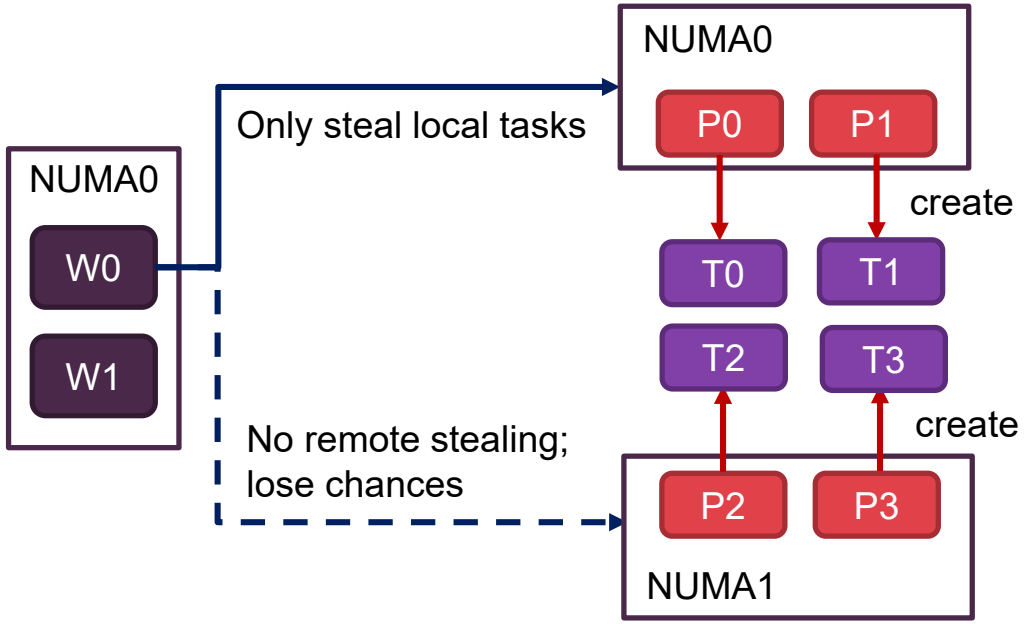
- broadwell node 36 cores (2 sockets)
- all src and dest buffer (64KB) are located at NUMA 0



Work-Stealing Strategy

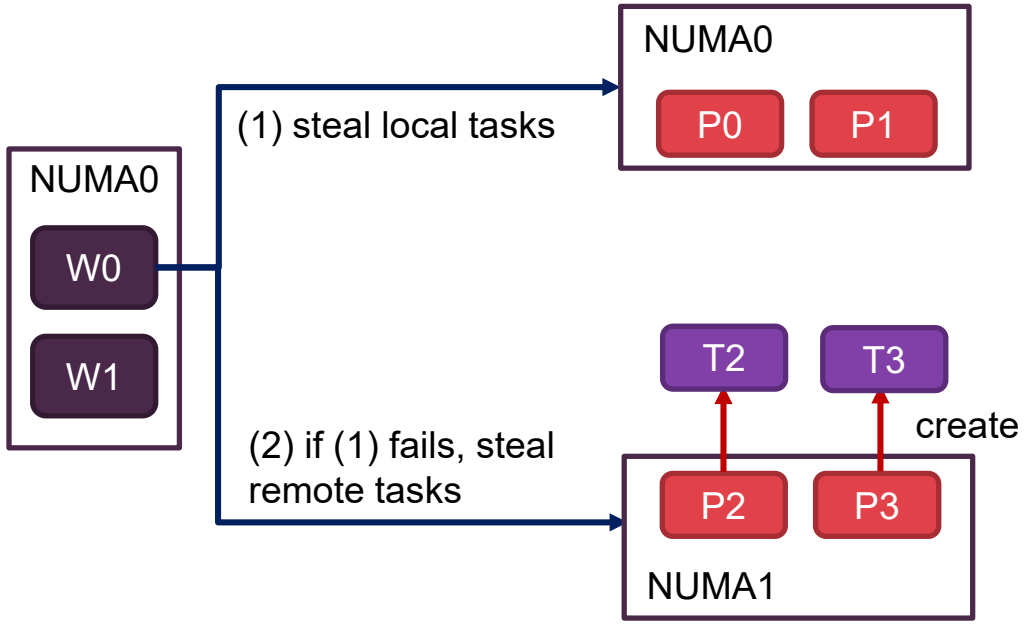
Localized Stealing

- workers can only perform local stealing
- victim is selected in a random way
- achieve best throughput when local workers saturate bandwidth
- **lose the chances of stealing remote tasks**



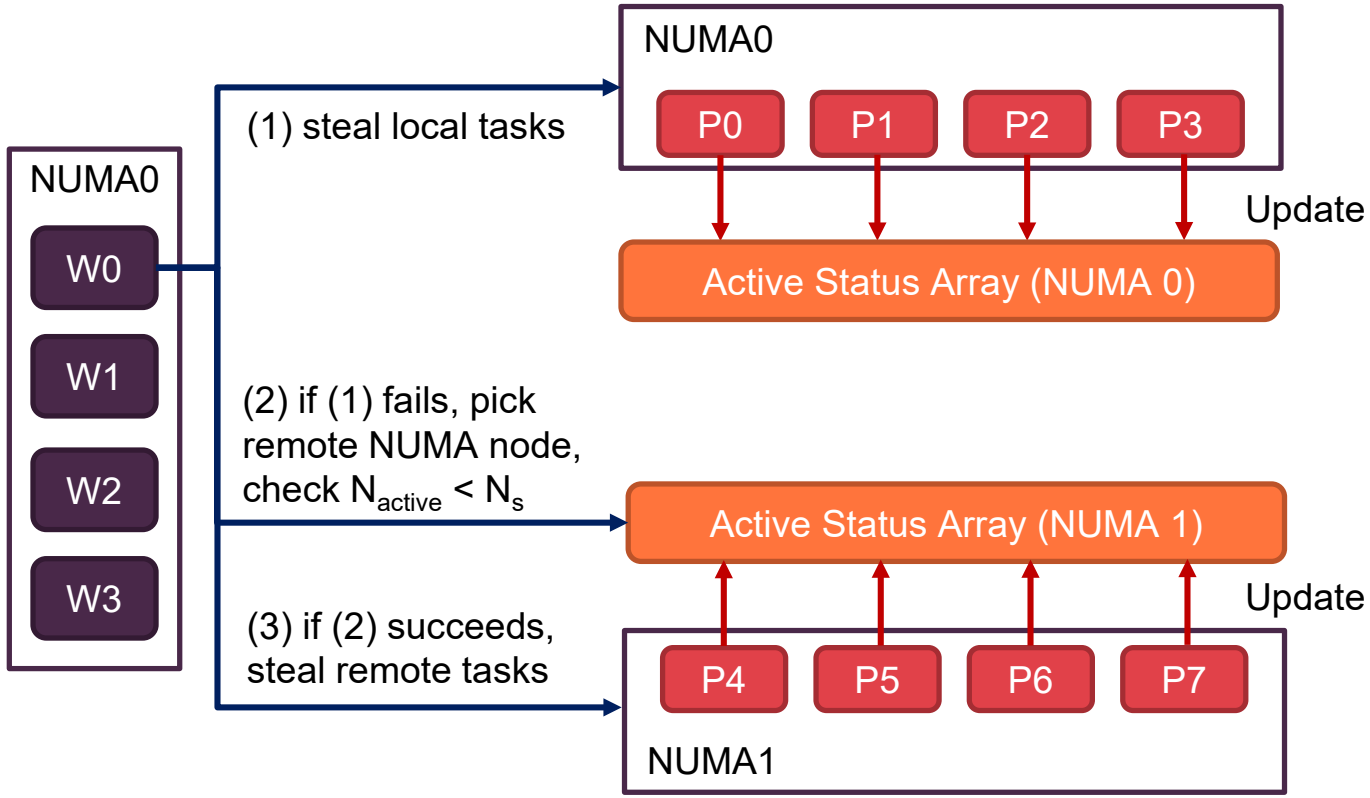
Mixed Stealing

- to catch the remote stealing chances
- workers steal local tasks in the beginning
- if fails, search and steal remote tasks next time
- improve performance when memory bandwidth is not saturated
- **may degrade performance for saturated situation**



Work-Stealing Strategy

Throughput-Aware Stealing



Quantify Bandwidth Utilization

- factors: datatype, buffer location, buffer size, task type and so on
- conservative method: define threshold N_s based on memcpy throughput profiling

Active Status

- if a process is processing a task, set active status as 1; otherwise 0
- used to check whether bandwidth is saturated or not

Optimization

Reversed Task Enqueue

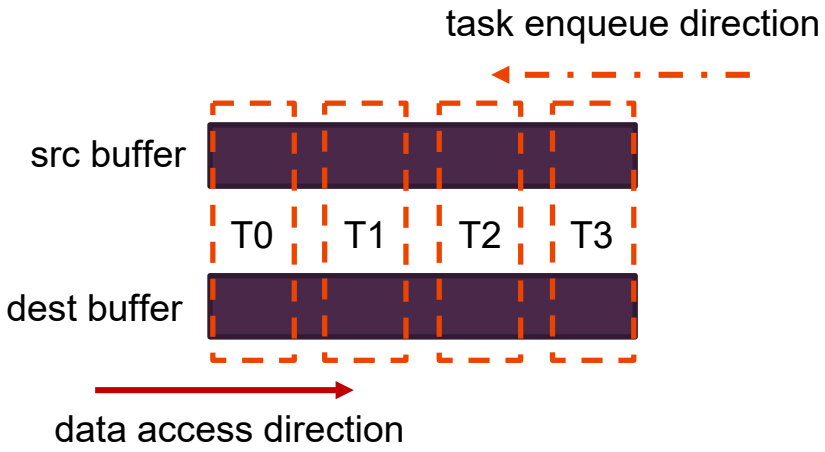
- receiver usually access received data after communication
- increase post-communication **data access cache reuse**

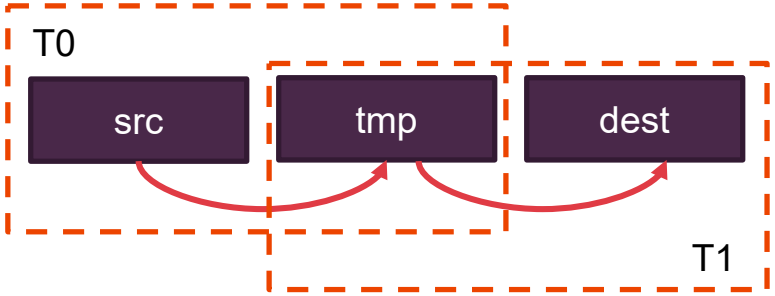
Noncontiguous Task Bundle

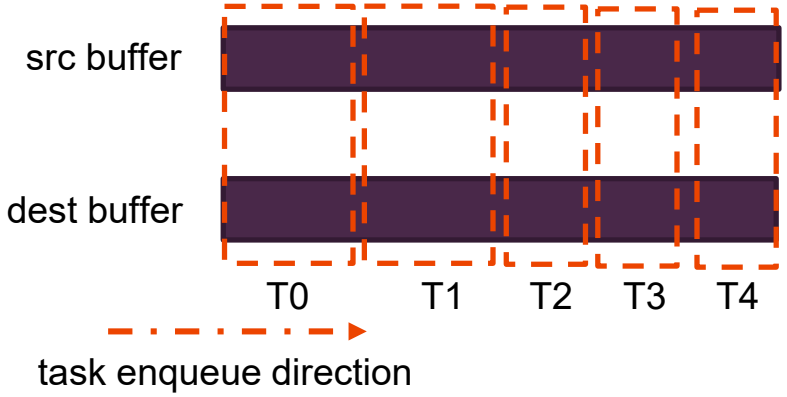
- require tmp buffer to hold intermediate results
- T0 and T1 should be stolen by the same worker to reuse tmp buffer
- **achieve tmp buffer cache reuse**

On-demand Chunking

- task chunk size should match buffer size
- **small chunk size increases parallelism**
- **large chunk size reduce task creation overhead**







Experiment Platform

Hardware Platform

Broadwell Node
- 18 cores/socket, 2 sockets in total
- 128GB DDR4
- L1 32KB, L2 256KB, L3 45MB

KNL Node
- 16 cores/socket, 4 sockets in total (SNC-4 cache mode)
- 96GB DDR4, 16GB MCDRAM
- L1 32KB, L2 1MB

Network Interconnect
- Intel Omni-Path (OPA)

Software Platform

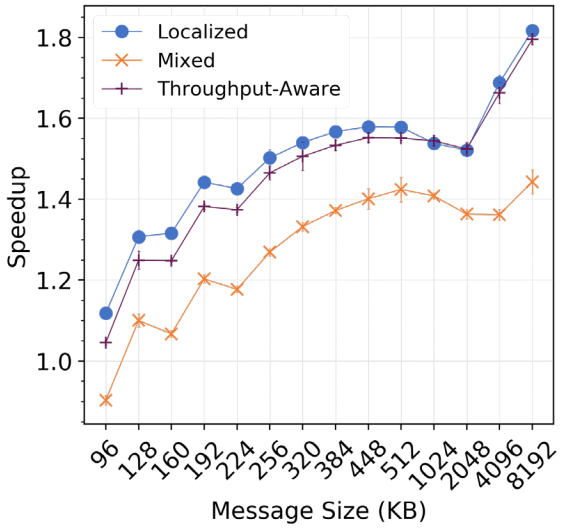
Baseline
- PIP-aware MPICH[1]
  - single-copy communication

CAB-MPI (Communication-AutoBalance MPI)
- PIP-aware MPICH + Work Stealing

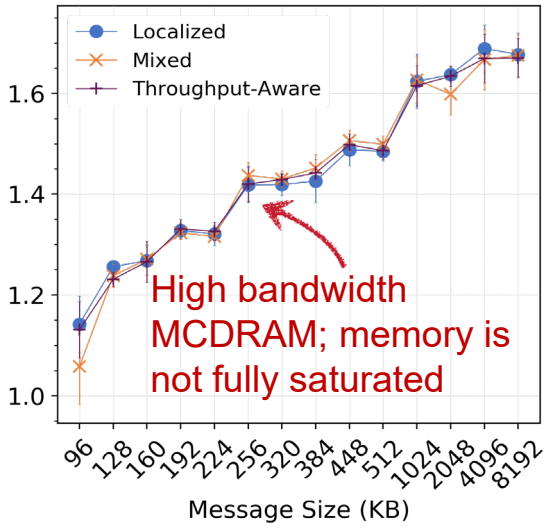
MPICH-xpmem-coop
- XPMEM-based pt2pt
- sender and receiver perform (cooperative) copy

[1] Hori, Atsushi, et al. "Process-in-process: techniques for practical address-space sharing." Proceedings of the 27th International Symposium on High-Performance Parallel and Distributed Computing. 2018.

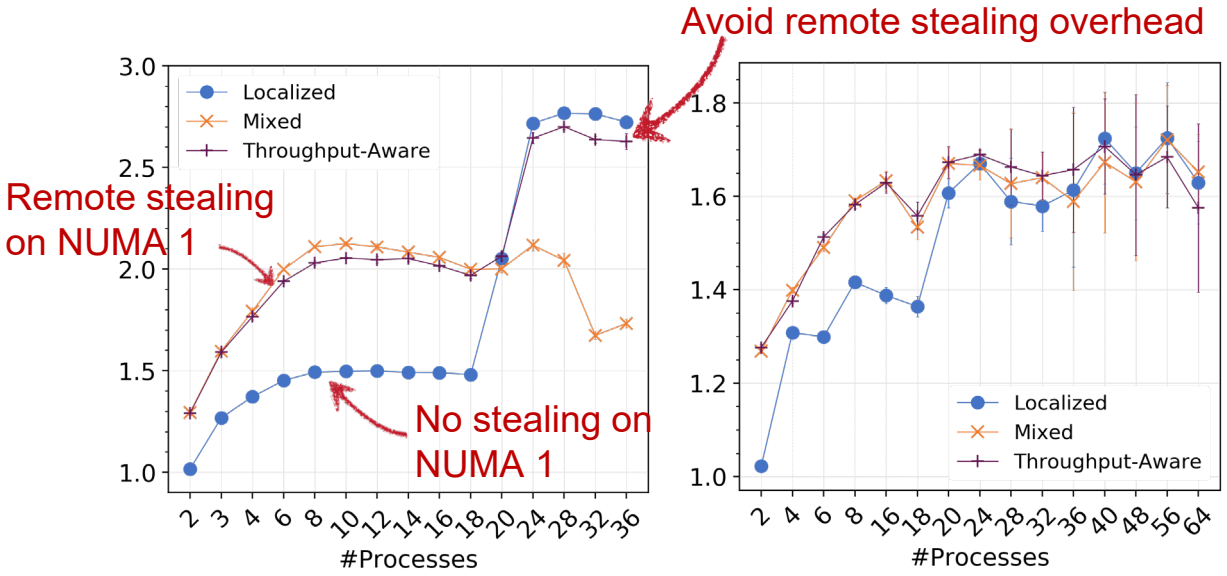
Microbenchmark – PingPong (contiguous data)



Broadwell intra-NUMA
(receiver creates tasks)



KNL intra-NUMA
(receiver creates tasks)



Broadwell inter-NUMA

KNL inter-NUMA

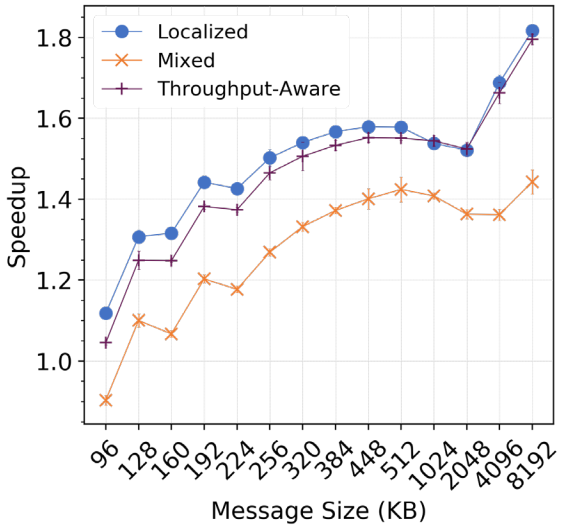
Message Size Based Test

- allocate 36/64 processes on Broadwell/KNL node
- rank 0 and 1 perform intra-NUMA pingpong
- other processes are idle

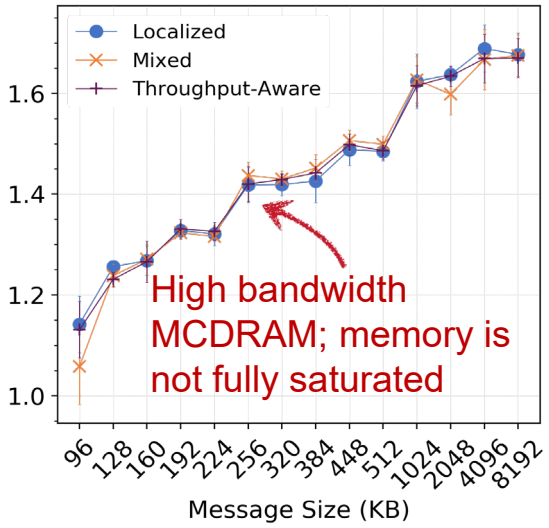
Process Based Test

- message size is fixed at 8MB
- rank 0 and 1 are on NUMA 0 and 1 respectively
- processes are filled from NUMA 0 to 1

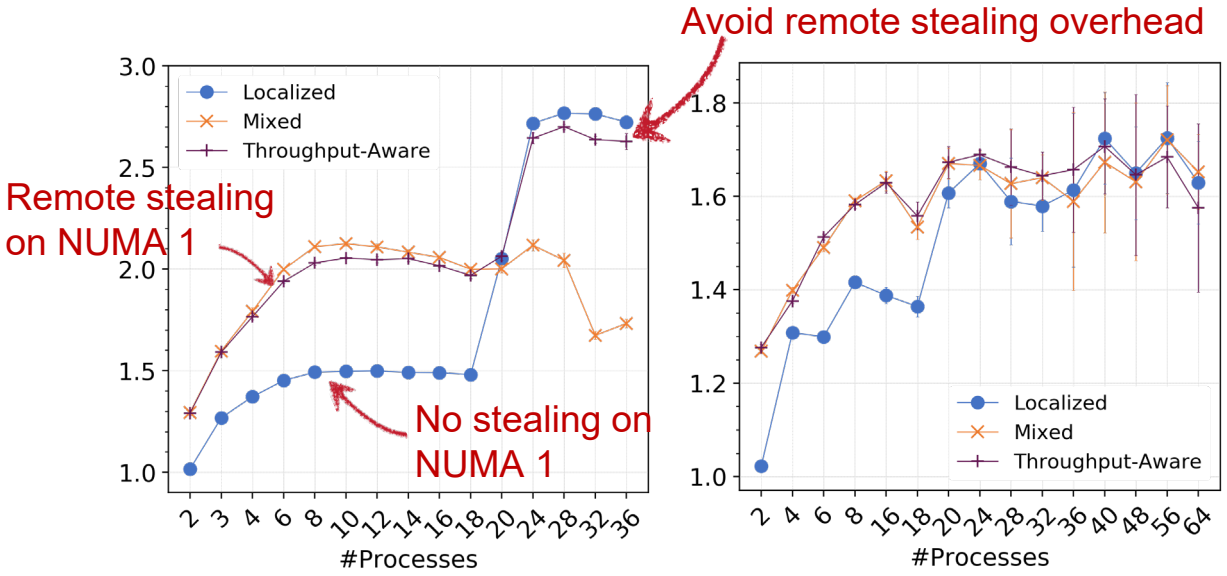
Microbenchmark – PingPong (contiguous data)



Broadwell intra-NUMA
(receiver creates tasks)



KNL intra-NUMA
(receiver creates tasks)



Broadwell inter-NUMA

KNL inter-NUMA

Message Size Based Test

- allocate 36/64 processes on Broadwell/KNL node
- rank 0 and 1 perform intra-NUMA pingpong
- other processes are idle

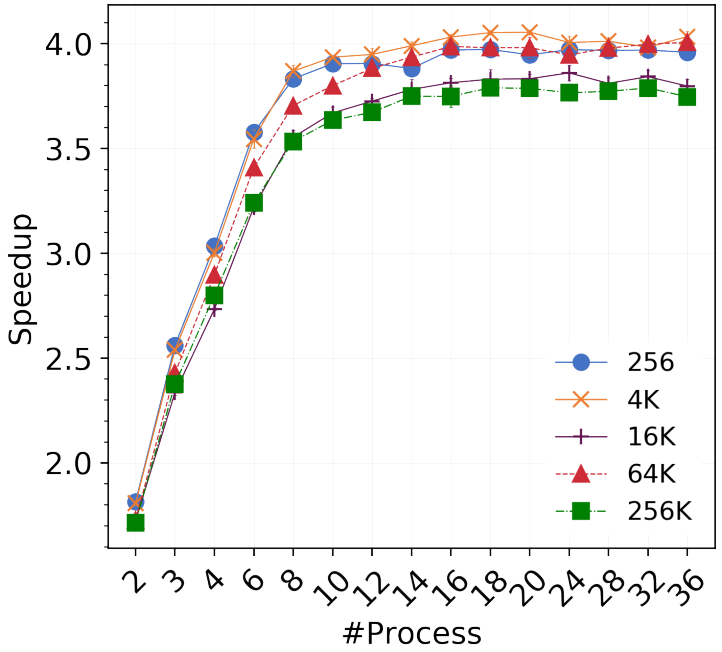
Process Based Test

- message size is fixed at 8MB
- rank 0 and 1 are on NUMA 0 and 1 respectively
- processes are filled from NUMA 0 to 1

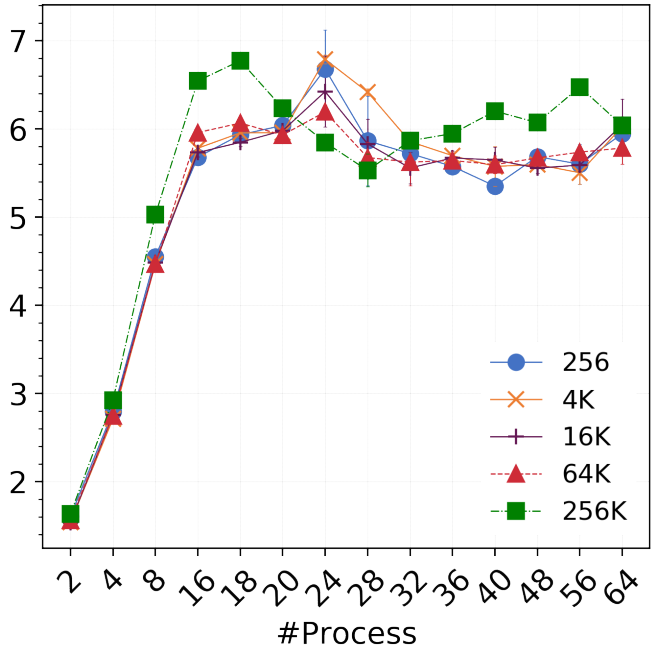
Conclusion

- throughput-aware strategy is on average better than localized and mixed strategy

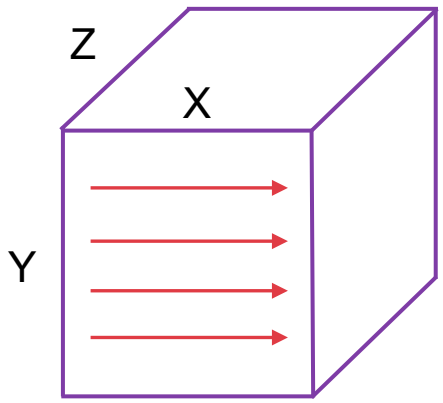
Microbenchmark – PingPong (Noncontiguous data)



Intra-NUMA Pack on Broadwell



Intra-NUMA Pack on KNL



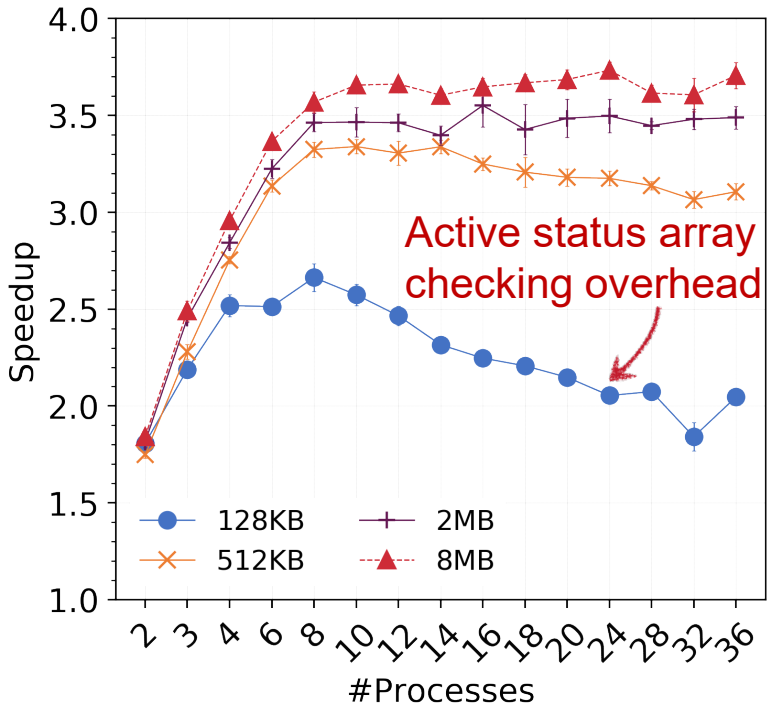
3D Matrix

- X as leading dimension
- element type: double (8 bytes)
- XZ vector datatype
  - block length x
  - stride x*y
  - count z

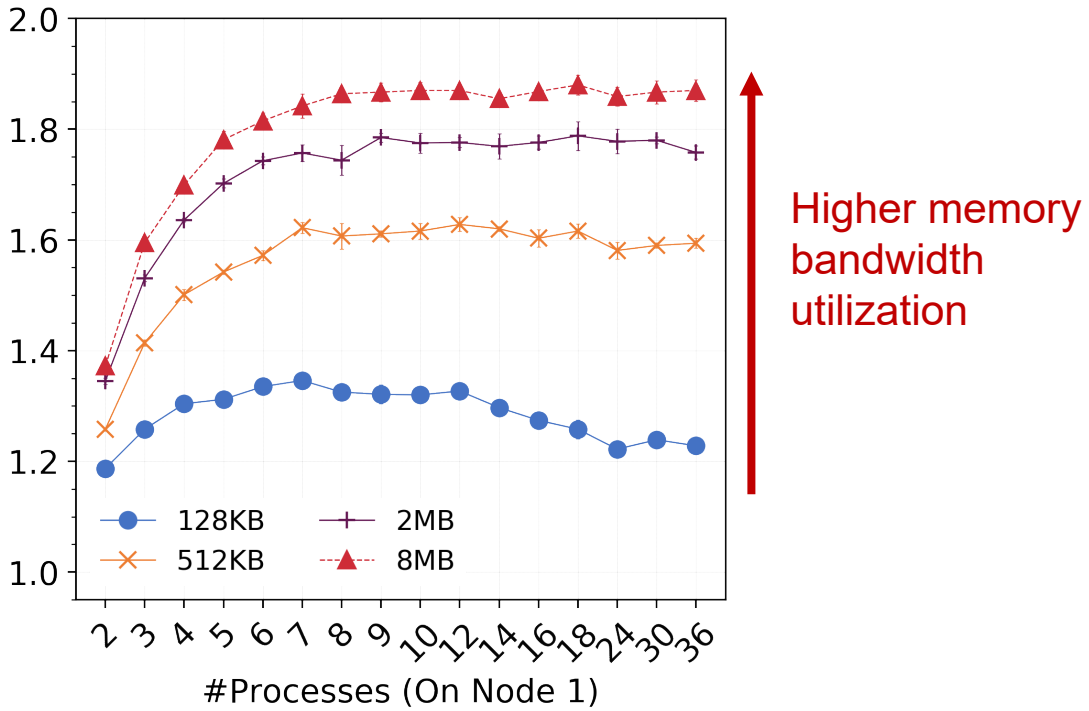
Pack-based PingPong

- XZ face data layout defines vector datatype
- matrix size x*y*z*sizeof(double)=1GB
- y is fixed at 2

Microbenchmark – RMA Accumulate (MPI_SUM)



Intra-node Accumulate
(origin process creates task)

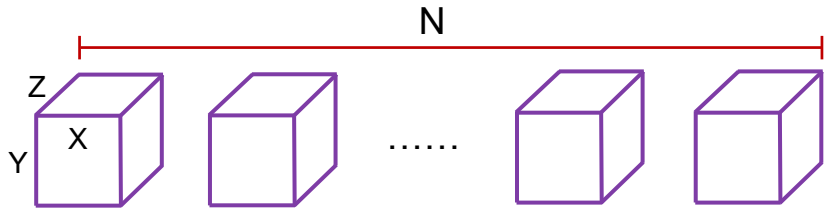


Inter-node Accumulate
(target process creates task)

- origin and target are on NUMA 0
- processes are filled from NUMA 0 and NUMA 1

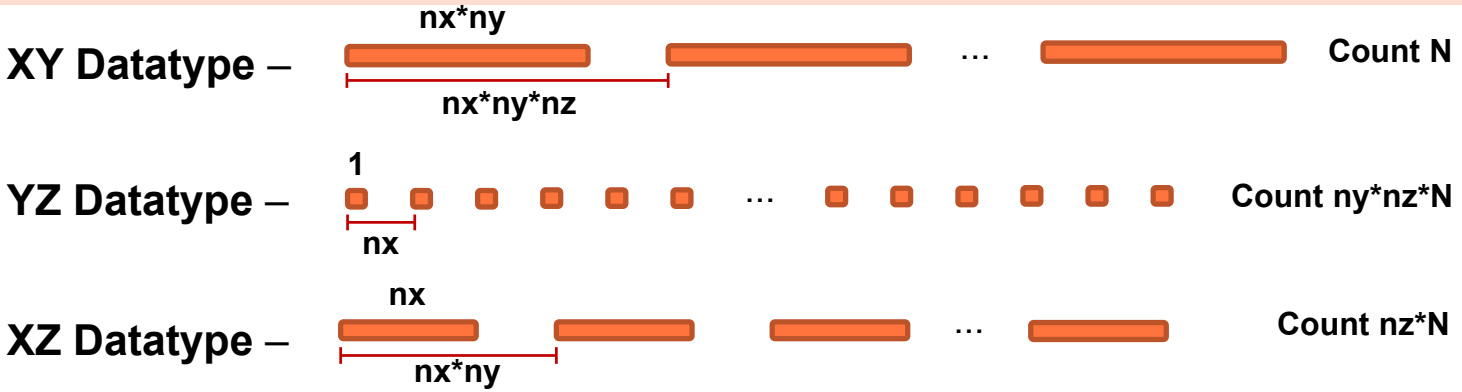
Minighost

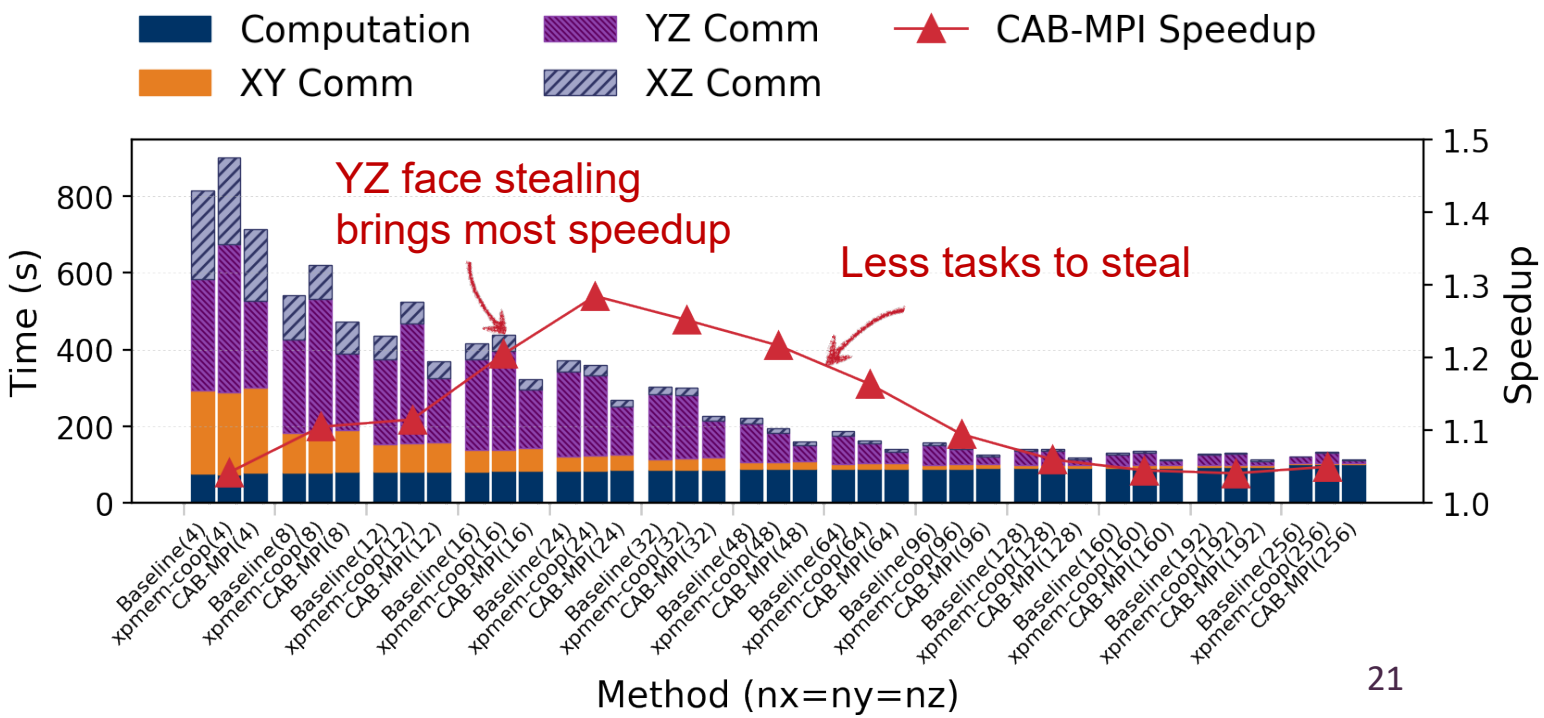
- 3D 7-point stencil
- each process owns ***N number of*** nx*ny*nz 3D matrix
- N*nx*ny*nz*sizeof(double) = 1GB



3D Matrix

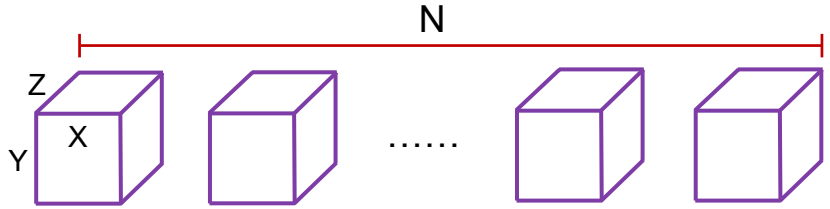
- X as leading dimension
- element type: double (8 bytes)





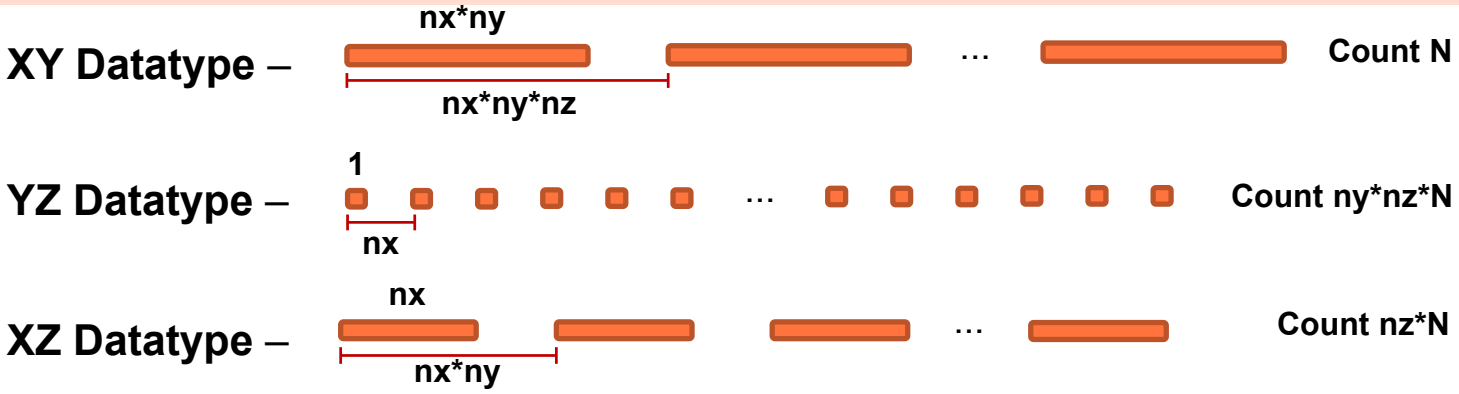
Minighost

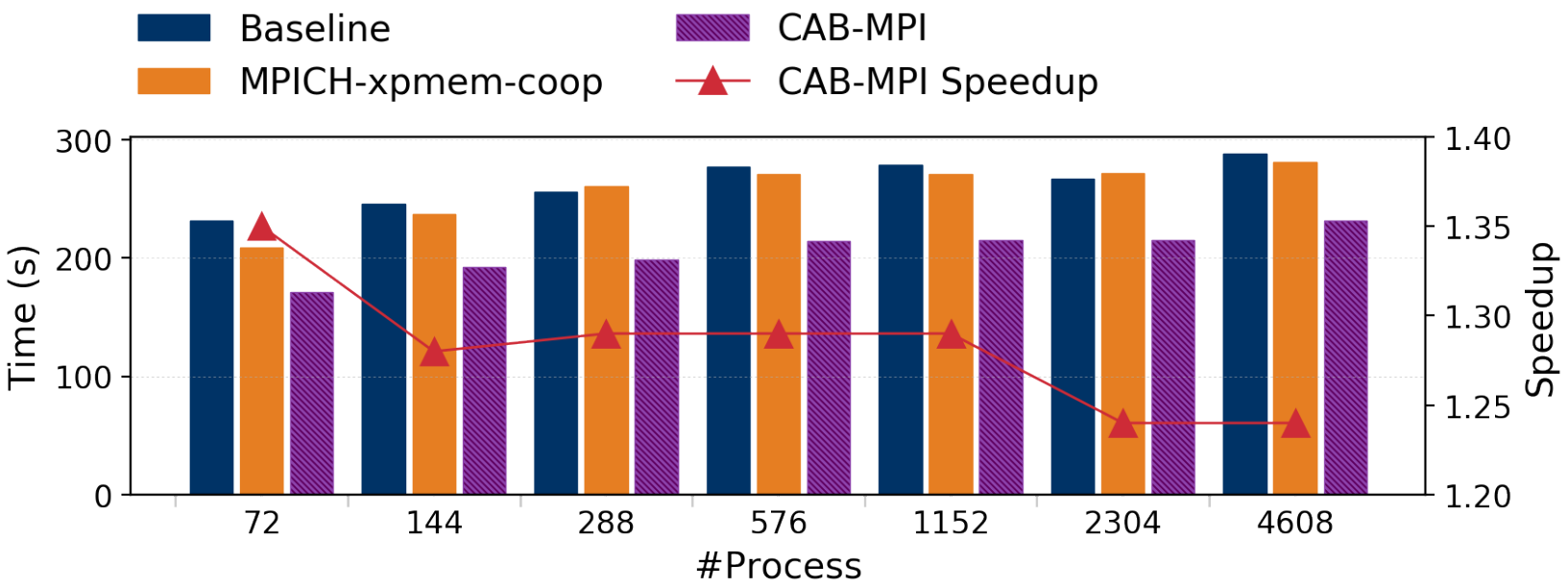
- 3D 7-point stencil
- each process owns ***N number of*** nx*ny*nz 3D matrix
- N*nx*ny*nz*sizeof(double) = 1GB



3D Matrix

- X as leading dimension
- element type: double (8 bytes)





Nwchem Proxy Application

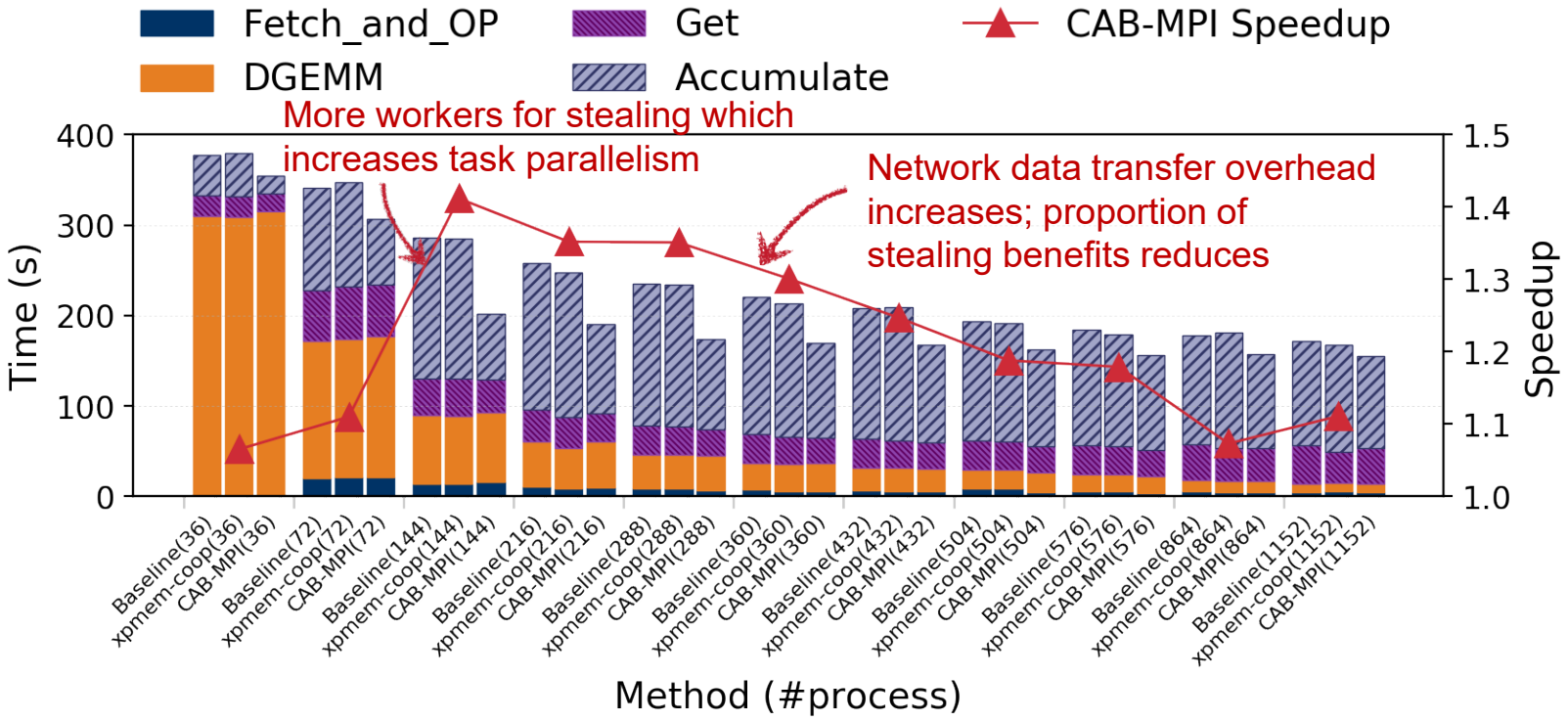
- Nwchem is widely used computational chemistry application suite
- Block-Sparse Matrix Multiplication (BSPMM) applies get-compute-accumulate pattern
- Overall workload is fixed; submatrix blocks are equally distributed among processes
- Irregular communication pattern brings more speedup

get-compute-accumulate pattern

get(A_i) get(B_i)

C_i = dgemm(A_i, B_i)

accumulate(C_i)



Summary

Communication imbalance widely exists in HPC applications

- MPI runtime
  - communication can happen among inter-node, intra-NUMA and inter-NUMA
- application
  - equally distributing workload is hard or impossible in some cases
- cause process idleness and performance loss

CAB-MPI: efficiently mitigate communication imbalance

- PiP shared memory technique brings lightweight interprocess data access
- throughput-aware strategy maximizes memory bandwidth utilization

Minighost and BSPMM results show the maximal 1.4x speedup and good scalability

Future Work

- explore and optimize MPI collective communication imbalance

Thank You